

Simulation of DACH-Style Detector-Actuated Traffic Signal Control in SUMO

A Python-Based Platform for LISA+[®] Logic with Loop Detectors, Push Buttons, Radio Beacons and R09-Telegrams

Markus Runhao Zhou^{1,*} , Johann Krüger¹ , Meng Wang¹ 

¹Chair of Traffic Process Automation, Technische Universität Dresden, Dresden, Germany

*Correspondence: Markus Runhao Zhou, runhao.zhou@tu-dresden.de

Abstract: In the DACH region (Germany, Austria, and Switzerland), traffic signal control is commonly implemented using LISA+[®] signal logic files that process inputs from traffic detectors and public transport priority systems. However, reproducing such control systems in SUMO typically requires high manual effort, as the signal logic must be reimplemented at a low level via the standard TraCI interface or must rely on GUI-based tools. This reimplementation often introduces simplifications or approximations, reducing simulation fidelity and limiting the transferability of results across different scenarios and control setups. We present a SUMO- and Python-based platform that enables the direct import of LISA+[®] signal control logic and the integration of LISA+[®]-compatible detector and telegram inputs into SUMO simulations. The platform directly uses LISA+[®] files and maps their signal groups, phase structures, and timing logic to the SUMO traffic signal controller. To drive the imported logic, a traffic sensor input layer maps simulated traffic data in SUMO to the required detector events and message inputs, including occupancy and flank/pulse events at the SUMO simulation resolution. By reproducing both the original signal logic and its controller-level sensor inputs, the proposed platform enables high-fidelity and reproducible simulation of DACH-style and LISA+[®]-based signalised intersections without reformulating the SUMO signal controller, supporting both research and engineering workflows.

Keywords: Traffic signal control, LISA+[®], SUMO, Detector modelling

1 Introduction

In the DACH region (Germany, Austria and Switzerland), traffic signal controllers at intersections operate with a well-established framework of engineering standards, legal regulations, tools and configuration files. Signal control logic executed by certified intersection controllers is typically based on LISA+[®] [1] signal logic files and is driven by a rich set of traffic sensor inputs. These components define not only the control logic,

but also the interaction between traffic demand, detector events and control decisions. Faithfully reproducing LISA+[®]-based signal control systems in the DACH region therefore requires both the original control logic and the corresponding controller-level input interfaces.

The default signal controllers provided by SUMO [2] have limited capability to represent event-driven inputs. Moreover, existing SUMO-based studies typically reimplement traffic signal control logic using the TraCI interface or simplified signal programs created with the network editor *netedit* of the SUMO software suite. This process is time-consuming and error-prone, and it often leads to discrepancies between simulated behaviour and real-world signal controller operation.

Several tools have been developed to enable the execution of high-fidelity signal control logic in SUMO. The first is *LiSuM* [3]–[5], which allows LISA+[®] signal logic to be executed within SUMO. However, *LiSuM* is provided as a GUI-based tool, and offers only limited access to simulation and controller states via the TraCI interface, which restricts its applicability in detailed data analysis and extended experimentation. Moreover, it cannot support R09-telegrams [6] for public transport priority. The second tool, *ocit2SUMO* [7], translates actual signal control logic files into SUMO traffic signal control definitions. While this tool maintains phase structure and signal timing, it provides only limited support for detector-based inputs and cannot capture the event-driven behaviour of the original controller. The third tool, developed within the *KI4LSA* project [8], enables white-box execution of LISA+[®] logic in SUMO. However, this tool also cannot support R09-telegrams. Overall, existing solutions limit the fidelity of simulation-based signal control systems and complicate the integration of certified signal control logic in both research and engineering workflows.

To address these challenges, we present a SUMO- and Python-based platform that enables the direct execution of LISA+[®] signal control logic together with LISA+[®]-compatible controller-level detector and telegram inputs, while making both simulation data and controller outputs available through the TraCI interface for further data access and analysis. Building on the basis developed in the *KI4LSA* project [8], our platform imports LISA+[®] files, and maps their signal groups, phase structures and timing logic to SUMO traffic signal controllers. In addition, we provide a traffic sensor input layer that generates loop-detector states and state transitions, pedestrian push button requests, radio beacon contacts and R09-telegrams from the SUMO simulation. This allows the original control logic to be driven by the same types of inputs it would receive at a real intersection, and to return control commands in the *OCIT*[®]-O [9] format described in Section 2.

By combining LISA+[®] logic with controller-level traffic sensor inputs, our platform allows SUMO to reproduce LISA+[®]-based signal control systems in the DACH region without redesigning or rewriting the controller for SUMO simulations. The resulting environment supports high-fidelity testing of signal control logic, detector configurations and public transport priority mechanisms in a fully reproducible and configurable setting. This enables both researchers and practitioners to evaluate traffic signal control under conditions that closely reflect real-world DACH intersections while retaining the flexibility of microscopic simulation.

The remainder of this paper is organised as follows. Section 2 introduces the relevant background and terminology. Section 3 describes how the original LISA+[®] traffic signal control logic is executed within SUMO. Section 4 presents the modelling of

LISA+[®]-compatible traffic sensor inputs in SUMO, covering loop-detector states and transitions, pedestrian push buttons, radio beacon contacts and R09-telegrams. Section 5 compares our platform with *ocit2SUMO*. Section 6 discusses the applications of the introduced platform. Finally, Section 7 summarises this work and outlines directions for future research.

2 Preliminary

To facilitate understanding of this platform, we introduce the essential terminologies of DACH traffic signal control, signal controller communication protocol *OCIT*[®]-O used in the DACH region, the relevant protocol parameters and attributes, the commonly used detector types and the R09-telegrams employed for public transport priority.

Terminology

We introduce two key terms: *signal groups* (SG), and *signal time plans* (STP; or *SZP* in German). In the DACH region, traffic signal control is organised at a fine granularity based on traffic movements. At an intersection, lanes are grouped together and controlled by an SG. Multiple SGs are then combined into an STP according to the conflict relations between movements and the required intergreen times, including amber, red–amber and start intervals.

This SG-based control closely matches the signal control mechanism in SUMO, which is based on the attribute connections. As illustrated in Fig. 1, consider an intersection where a right-turn lane on the north approach connects to two outgoing lanes on the west approach. This right-turn movement is typically controlled by a single SG in the DACH region, for example denoted as K1. In SUMO, the same movement is represented by two separate connections, corresponding to the right-turn into the first (connection ID: 0) and second (connection ID: 1) westbound outgoing lanes. Each connection has a unique ID in SUMO.

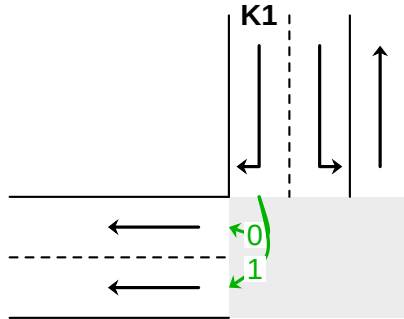


Figure 1. The right-turn movement controlled by SG K1 is represented in SUMO by two connections (0 and 1) to the westbound outgoing lanes.

To execute LISA+[®] signal logic in SUMO, we therefore generate a configuration file that maps each LISA+[®] SG to the corresponding set of SUMO connections. An example of this mapping is shown in Tab. 1.

Table 1. Mapping between SGs in LISA+[®] and SUMO connections.

Signal group	connection IDs in SUMO
K1	0, 1, 2, 3
K3	4, 5, 6
K4	7, 8

OCIT[®]-O protocol

OCIT[®]-O (Open Communication Interface for Road Traffic Control Systems-Outstations: Traffic signal controllers) [9] defines the standardised mechanisms, parameters and attributes used for the operation and communication of traffic signal controllers in the DACH region. It defines the representation of signal control logic, and external control data exchanged between intersection controllers and external systems. The parameters and attributes used in LISA+[®] signal logic files follow the OCIT[®]-O protocol, which makes it the common execution and communication standard for DACH traffic signal control systems.

Relevant protocol attributes and parameters

We introduce two key OCIT[®]-O protocol attributes, as summarised in Tab. 2. The signal control commands generated by the LISA+[®] logic are numerical codes defined by the OCIT[®]-O protocol for each SG. These numerical codes must be translated into SUMO traffic signal state definitions to apply the LISA+[®] control commands in the simulation. The mapping is summarised in Tab. 3.

Table 2. Key attributes defined in the OCIT[®]-O protocol.

Attribute	Definition
OCIT [®] -O signal colour	Numerical code of signal state.
User programme value (AP-Werte in German)	User programme values exchanged between the intersection controller and external systems for modifying and querying operational states.

Table 3. Mapping between OCIT[®]-O signal colour codes and SUMO traffic signal states.

OCIT [®] -O signal colour code	Signal state	SUMO signal state definition
48	Green	g or G
3	Red	r
12	Amber	y
15	Red-amber	u
8	Amber blinking	o
0	Dark	r, o, 0 or s

The mapping between OCIT[®]-O signal colour codes and SUMO traffic signal states is not one-to-one, and depends on the operational context of the intersection. For example, the code 48 represents green in OCIT[®]-O, but it cannot distinguish between protected and permissive movements. In SUMO, these are represented by priority green

(G) and non-priority green (g), respectively. Consequently, a single $OCIT^{\text{®}}-O$ value 48 may correspond to different SUMO signal states depending on whether the movement has priority or is subject to yielding.

Similarly, the $OCIT^{\text{®}}-O$ value 0 represents a dark signal. In DACH signal control practice, the dark state may correspond to different operational modes, such as a full prohibition of movement or conditional turning under visual observation. These distinctions are not encoded in the $LISA^{\text{®}}$ logic, and therefore cannot be inferred from the numerical value alone. Therefore, users must specify the appropriate SUMO signal state for each SG based on the operational configuration of the simulated intersection.

After applying this mapping, the signal control commands generated by the $LISA^{\text{®}}$ logic are executed in SUMO using the default TraCI function `traci.trafficlight.setRedYellowGreenState()`. For example, consider the intersection in Tab. 1, which is controlled by three SGs K1, K3 and K4. If K1 is green and the other two SGs are red, the $LISA^{\text{®}}$ controller outputs the command 48/3/3. Based on the mapping, this is translated into the SUMO traffic signal state string `ggggrrrrrr`.

Controller-level detector inputs

This work models the controller-level detector inputs required by the imported $LISA^{\text{®}}$ logic. In the considered DACH-style control setups, these inputs include loop-detector states and state transitions, pedestrian push-button requests and radio beacon contacts. Their corresponding representations in SUMO are summarised in Tab. 4.

Table 4. Mapping between controller-level detector inputs and their representations in SUMO.

Controller-level input	Representation in SUMO	Deployment in SUMO
Loop-detector input (<i>Schleifendetektor</i> in German)	E1 or short E2 detector	Defined directly in the network editor <i>netedit</i> .
Radio beacon contact (<i>Kontakt</i> in German)	E1 or E2 detector	
Pedestrian push button (<i>Taster</i> in German)	N/A	Implemented via TraCI pedestrian functions.

Other physical sensing technologies, such as camera-based, radar-based or infrared detection, are not modelled as separate sensor hardware classes in this work. They can be represented if their outputs are available as controller-level detector states or events compatible with the $LISA^{\text{®}}$ logic. For loop-detector inputs, the platform uses both detector states and state transitions. Rising and falling edges can therefore be generated at the SUMO simulation resolution and passed to the imported $LISA^{\text{®}}$ logic, where flank-based detector behaviour is required.

For radio beacon contacts, which are primarily used for public transport vehicle detection, we recommend the use of E2 detectors. In the considered configurations, radio beacon contacts generate an occupied signal lasting approximately 3-4s, and the spatial coverage of E2 detectors better reflects this behaviour than the point-based E1 detectors.

R09-telegram

In the DACH region, traffic signal controllers can receive telegrams from public transport vehicles via radio beacon systems (*Funkbakensystem* in German). When a vehicle passes a radio beacon contact, a telegram is transmitted to the intersection controller. This telegram contains the identifier of the radio beacon, the public transport vehicle line, the destination, the deviation from the scheduled timetable and a priority level. The format of these messages is standardised as the R09-telegram [6].

Table 5. Structure and example of a R09-telegram for public transport priority.

Telegram field	Definition	Example value
Request ID (<i>Meldepunkt</i> in German)	Identifier of the radio beacon that detected the vehicle	640187
Line ID (<i>Linie</i> in German)	Public transport line identifier	84
Destination (<i>Ziel</i> in German)	Destination identifier of the vehicle	11
Schedule deviation (<i>Fahrplanabweichung</i> in German)	Deviation from the scheduled arrival time in seconds	21
Priority (<i>Priorität</i> in German)	Priority level of the request	1

Based on the received R09-telegram, the signal control logic determines appropriate public transport priority actions, such as green extensions. In our platform, these telegrams are generated from the simulation and processed by the LISA+[®] logic in the same way as at a real intersection. Tab. 5 illustrates the structure of an R09-telegram using an example. In this example, a bus on line 84, heading towards destination 11, is detected by radio beacon 640187 with a schedule deviation of 21s and a priority level of 1, which typically represents normal priority. The corresponding R09-telegram is encoded as 640187/84/11/21/1, representing the fields *Request ID*, *Line ID*, *Destination*, *Schedule deviation* and *Priority*, respectively.

3 Executing LISA+[®] Logic in SUMO

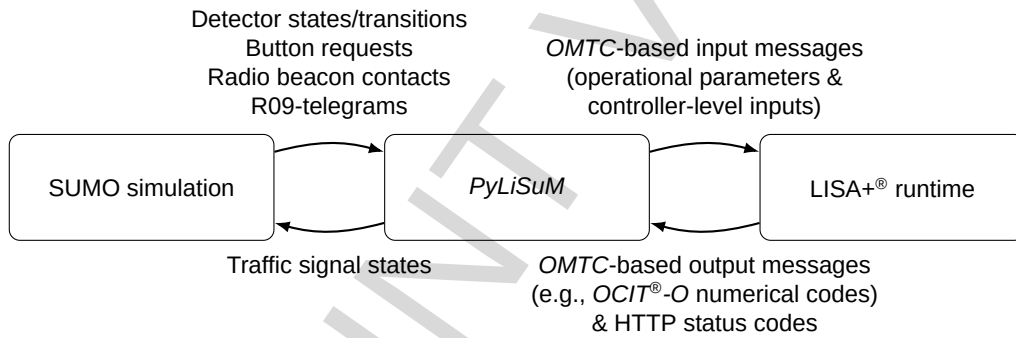
To execute LISA+[®] logic in SUMO, five components must be configured, as summarised in Tab. 6. Together with the SUMO simulation, the overall architecture can be separated into three parts: the SUMO simulation, the configuration and middleware layer and the LISA+[®] runtime, as shown in Fig. 2. The LISA+[®] runtime comprises the logic files, the *OmlFgServer* and the *OMTC* service endpoints. The configuration and middleware are implemented together in Python and are referred to as *PyLiSuM*.

The overall architecture and operational workflow can be treated as a bidirectional transmission and translation process between the SUMO simulation and the LISA+[®] runtime via *PyLiSuM*. All information required by the simulation and the runtime is exchanged and translated within *PyLiSuM*.

LISA+[®] logic files typically contain multiple signal control modes and operation patterns. Traffic engineers define different modes to accommodate peak and off-peak periods, special events, weekdays and weekends, as well as intersections with or without public transport priority. Prior to execution, six operational parameters must therefore be specified, as summarised in Tab. 7.

Table 6. Essential components for executing LISA+[®] signal control logic in SUMO.

Component	Purpose	Source
Logic files (e.g., z*_fg*.jar, z*_fg*.xml)	Signal control logic	Exported from the LISA+ [®] software.
Configuration file (e.g., *.xml or *.yaml)	Mapping between SUMO and LISA+ [®] objects	Defined by users to map detector names between SUMO and LISA+ [®] logic files, and to associate SGs in the logic files with the attribute connections.
Middleware (e.g., Python environment, traffic sensor input layer)	Starts the <i>OmlFgServer</i> , formulates controller-level detector inputs, button requests, beacon contacts and R09-telegrams as LISA+ [®] OMTC input messages, and applies the resulting signal control to SUMO	This platform.
<i>OmlFgServer</i>	OMTC-based execution interface for LISA+ [®]	Part of the LISA+ [®] software suite.
OMTC service endpoints	LISA+ [®] communication endpoints	

**Figure 2.** Architecture for executing LISA+[®] signal logic in SUMO.**Table 7.** Parameters for defining the LISA+[®] signal control mode and operation pattern in SUMO.

Parameter	Definition
controlMode	Weekday control mode, local clock-based control mode, or other user-defined control modes.
coordinated	Indicates whether the signal logic operates in coordination with other intersections.
sp	Signal programme identifier.
va	Abbreviation of <i>Verkehrsabhängig</i> in German, indicating whether traffic-actuated control is enabled.
oev	Abbreviation of <i>Öffentlicher Verkehr</i> in German (often written as <i>oev</i>), indicating whether public transport priority is enabled.
iv	Abbreviation of <i>Individueller Verkehr</i> in German, indicating whether private vehicle priority is enabled.

The OMTC-based input and output messages follow fixed structures, as shown in Fig. 3. The input message consists of seven fields, and the output message consists of six fields. Except for the first field, which contains the current simulation step, all remaining fields are enclosed in curly braces {}. An overview of the input message

fields and their internal structures is provided in Tab. 8, while Tab. 9 summarises the output message fields.

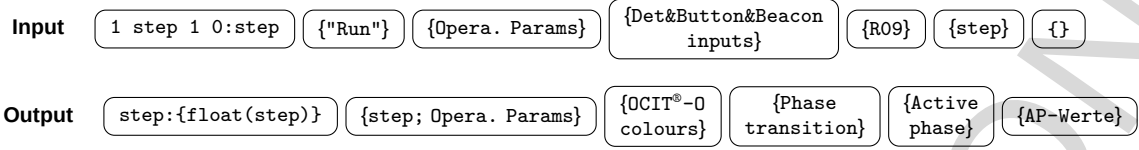


Figure 3. Structure of OMTC-based input and output messages.

Table 8. Structure of OMTC-based input messages with example.

Input message field	Structure	Example
Field 1 (Simulation step)	1 <step> 1 0:<step>	1 30 1 0:30, indicating that the current simulation step is 30.
Field 2 (Execution status)	{"Init"} or {"Run"}	{"Run"}, indicating that the LISA+ [®] logic is running.
Field 3 (Control parameters)	{controlMode;0;0;0;1;STP;1;va;iv;oev;coordinated}	{4;0;0;0;1;6;1;2;1;2;0}, indicating control mode 4, signal time plan 6, traffic-actuated control enabled, private vehicle priority disabled, public transport priority enabled and no coordination.
Field 4 (Detector, button and beacon inputs)	{{(0)<state>/ (1)<state>/...}}	{{(0)1/(1)-1/...}}, indicating that the first detector input is active or occupied and the second is inactive or unoccupied. All detectors, pedestrian buttons and radio beacon contacts are indexed starting from 0 according to the list defined in the LISA+ [®] logic files. The formulation of this field is described in detail in Section 4.
Field 5 (R09-telegram)	{R09-telegram}	{640187/84/...}, representing a standard R09-telegram.
Field 6 (Simulation step)	{<step>}	{30}, indicating that the current simulation step is 30.
Field 7 (Reserved)	{}	{}, empty but required by the OMTC message format.

Communication between *PyLiSuM* and the LISA+[®] runtime is implemented via RESTful services over HTTP [10]. Standard HTTP status codes, such as 200 (successful request) and 502 (gateway error), are used to indicate whether the transmitted OMTC-based messages are valid and successfully processed.

Once the SUMO simulation and all required execution components, including the *OmIFgServer* and the OMTC services, have been started and the necessary parameters and message formats have been configured, the SUMO simulation with LISA+[®] logic can be executed. The execution begins with a set of initialisation procedures. Five initialisation steps are performed. First, *SetDataDir* specifies the directory containing all required data. Second, *GetTaskList* and *RemoveTask* are used to query and clear any existing OMTC tasks, ensuring a clean execution context. Third, *SetTask* activates the OMTC service to receive OMTC-based input messages and to return output messages. Fourth, *GetObjectList* retrieves the objects defined in the LISA+[®] logic files. Finally, the LISA+[®]-SUMO mapping configuration file is loaded.

Table 9. Structure of OMTC-based output messages with example.

Output message field	Structure	Example
Field 1 (Simulation step)	<code><step>:{float(<step>)}</code>	30:{30.0}, indicating that the current simulation step is 30, and the second value is encoded as a floating-point number with one decimal place.
Field 2 (Control parameters)	<code>{0;<step>;1;STP;1;coordinated;va;iv;oev;1}</code>	{0;30;1;6;1;0;2;1;2;1}, indicating simulation step 30, signal time plan 6, no coordination, traffic-actuated control enabled, private vehicle priority disabled and public transport priority enabled. This field mirrors the corresponding parameters in the input message.
Field 3 (Signal colours)	<code>{OCIT[®]-0 colour code vector}</code>	{48/3/0/...}, indicating that the SG with index 1 is green, index 2 is red and so on, according to the ordering defined in the LISA+ [®] logic files.
Field 4 (Phase transitions)	<code>{phase transition vector}</code>	{0/0/0/0/1/...}, indicating that phase transition 5 is currently active.
Field 5 (Active phase)	<code>{phase vector}</code>	{0/0/1/...}, indicating that phase 3 is active. If a phase transition is active in Field 4, this phase 3 is approaching its termination.
Field 6 (AP-Werte)	<code>{<AP-Werte>}</code>	{394/3010/...}, representing the user programme values (AP-Werte) defined in the LISA+ [®] logic files and returned by the runtime.

After successful initialisation, the SUMO simulation with LISA+[®] logic enters a periodic execution mode, following the workflow illustrated in Fig. 2. The detailed runtime interaction and the corresponding TraCI function calls are shown in Fig. 4.

4 Modelling LISA+[®]-Compatible Detector and Telegram Inputs

Detector, button and beacon inputs are provided to the LISA+[®] logic via field 4 of the OMTC-based input messages. This field follows the structure `{(0)<state>/(1)<state>/...}`, as described in Tab. 8. This work models the controller-level input interface required by the imported LISA+[®] logic, rather than the physical sensing technology itself. Pulse-like and flank-based behaviour is represented by state transitions between consecutive input messages at the SUMO simulation resolution. All input devices, including single loop detectors, pedestrian push buttons and radio beacon contacts, are uniformly defined in the logic files. Each input device is assigned a unique index. Since the OMTC-based input field used by the runtime follows 0-based indexing, the indices of all input devices must be converted accordingly to ensure consistency with the LISA+[®] logic processing.

The controller-level inputs considered in this work, including single loop detector inputs, pedestrian crossing push button inputs, radio beacon contact inputs and R09-telegrams, are modelled based on the logic files of a representative signalised intersection in Leipzig, Germany.

Single loop detectors

Single loop detectors are inductive detectors installed in traffic lanes and are commonly used for vehicle presence detection in the considered DACH installations. In real-world

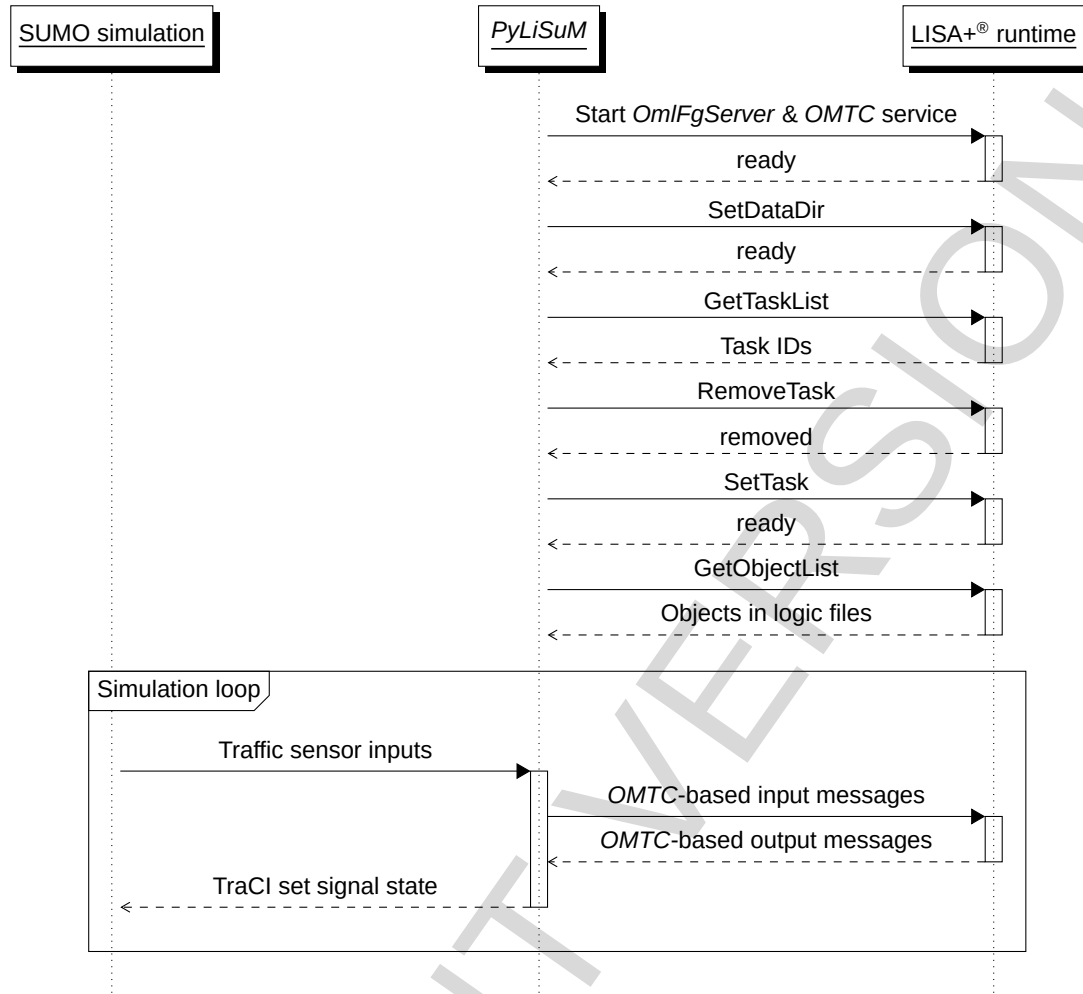


Figure 4. Initialisation and periodic interaction for executing LISA+® logic in SUMO. The last two interactions repeat once per simulation step.

installations, the detector electronics expose a binary controller-level input, often derived from high or low voltage levels, indicating whether a vehicle occupies the detection zone.

In SUMO, single loop detectors are modelled using E1 detectors, which are visualised as yellow squares, as shown in Fig. 5. When a vehicle passes over or occupies a detector, the detector state switches to 1. Otherwise, the state remains -1. The detector state is sampled at each simulation step and mapped to the corresponding input index in field 4 of the OMTC-based input message. Where the imported LISA+® logic uses flank-based conditions, rising and falling flanks are represented by changes between consecutive field-4 values at the SUMO simulation resolution. In this way, real-time controller-level detector information is provided to the LISA+® logic. For example, an input message field 4 represented as $\{(0)1/(1)-1/(2)-1/(3)1/\dots\}$ indicates that the single loop detectors with indices 0 and 3 are occupied, while those with indices 1 and 2 are not.

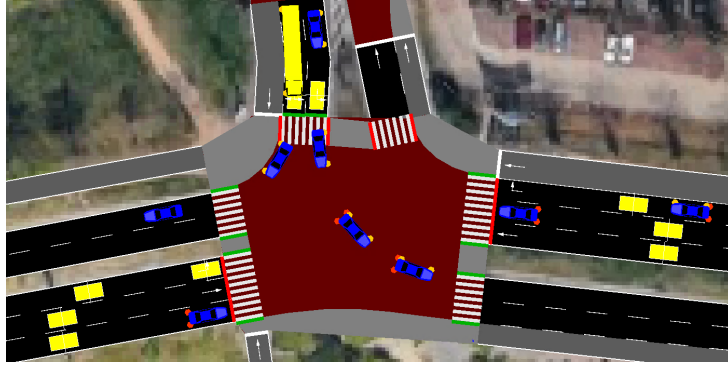


Figure 5. Single loop detectors modelled as E1 detectors in SUMO (yellow squares).

Pedestrian push buttons

Pedestrian push buttons are used to request pedestrian green at signalised crossings. In field applications, a button press generates a short binary pulse signal that is transmitted to the traffic signal controller, and stored until it is served by the signal logic.

In SUMO, pedestrian push buttons B are modelled as persistent binary input signals triggered by pedestrian arrival events. Once activated, the push button input is set to 1, and remains active until it is acknowledged and reset by the LISA+[®] logic. This modelling approach replicates the storage behaviour of pedestrian demand in real-world traffic signal controllers. This mechanism avoids the loss of short pulse signals in discrete simulation steps and ensures reliable processing of pedestrian requests.

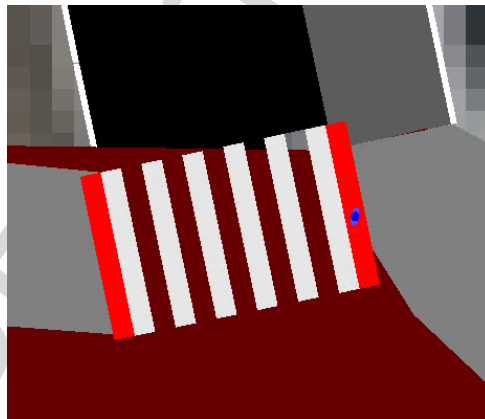


Figure 6. A pedestrian (blue point) waiting in the *walkingarea* (grey area to the right) before crossing the associated pedestrian crossing in SUMO.

However, pedestrian push buttons cannot be physically attached to crossings or walkingareas in SUMO. This limitation is not unique to SUMO and is shared by most existing traffic simulation software. Therefore, pedestrian push button inputs are generated using a combination of three TraCI functions:

- `traci.edge.getLastStepPersonIDs()`, which identifies pedestrians currently present at the intersection.
- `traci.person.getNextEdge()`, which determines whether a pedestrian is about to enter a specific crossing or is leaving the intersection.
- `traci.person.getWaitingTime()`, which indicates whether a pedestrian has arrived at a crossing, and is forced to wait for a green signal.

When pedestrians are present at an intersection and waiting to cross, as illustrated in Fig. 6, the combination of these three TraCI functions generates a binary pedestrian request signal. This binary input signal is then encoded according to the logic described in the pseudocode in Algorithm 1.

Algorithm 1 Pedestrian push button input generation using TraCI

Require: Set of walkingareas $\mathcal{W}$, set of crossings $\mathcal{C}$, set of pedestrian buttons $\mathcal{B}$, mapping $\text{ButtonsOnWhichSide}(w) : \mathcal{W} \rightarrow 2^{\mathcal{B}}$, mapping $\text{AllowedButtons}(c) : \mathcal{C} \rightarrow 2^{\mathcal{B}}$, previous pedestrian activity $\text{prevActive}[b]$, latched button states $\text{latched}[b]$ for all $b \in \mathcal{B}$, service indicator $\text{served}[b]$ from LISA+[®] logic for all $b \in \mathcal{B}$.

Ensure: Latched pedestrian push button inputs $\mathbf{u}(t) = \{u_b(t)\}_{b \in \mathcal{B}}$, where $u_b(t) \in \{-1, 1\}$.

```

1: function PedestrianButtonInputs
2:    $\text{rawActive}[b] \leftarrow -1 \quad \forall b \in \mathcal{B}$ 
3:   for all  $w \in \mathcal{W}$  do
4:     for all  $p \in \text{traci.edge.getLastStepPersonIDs}(w)$  do
5:       if  $\text{traci.person.getWaitingTime}(p) > 0$  then
6:          $c \leftarrow \text{traci.person.getNextEdge}(p)$ 
7:         if  $c \in \mathcal{C}$  then
8:            $\mathcal{B}_{\text{valid}} \leftarrow \text{ButtonsOnWhichSide}(w) \cap \text{AllowedButtons}(c)$ 
9:           for all  $b \in \mathcal{B}_{\text{valid}}$  do
10:             $\text{rawActive}[b] \leftarrow 1$ 
11:          end for
12:        end if
13:      end if
14:    end for
15:  end for
16:  for all  $b \in \mathcal{B}$  do
17:     $\text{rising} \leftarrow (\text{rawActive}[b] = 1) \wedge (\text{prevActive}[b] \neq 1)$ 
18:    if  $\text{served}[b] = 1$  then
19:       $\text{latched}[b] \leftarrow -1$ 
20:    else if  $\text{latched}[b] = 1 \vee \text{rising} = 1$  then
21:       $\text{latched}[b] \leftarrow 1$ 
22:    else
23:       $\text{latched}[b] \leftarrow -1$ 
24:    end if
25:  end for
26:   $\text{prevActive} \leftarrow \text{rawActive}$ 
27:  return  $\mathbf{u}(t) \leftarrow \{\text{latched}[b]\}_{b \in \mathcal{B}}$ 
28: end function

```

At each simulation step, pedestrians are identified on all walkingareas $\mathcal{W}$ of the intersection using TraCI. Only pedestrians that are currently waiting and whose next movement is directed towards a valid pedestrian crossing $c \in \mathcal{C}$ are considered. This ensures that pedestrian requests are generated exclusively for pedestrians who intend to cross.

For each such pedestrian, candidate push buttons are determined based on two predefined mappings. First, the mapping $\text{ButtonsOnWhichSide}(w)$ specifies the set of pedestrian push buttons that are physically accessible from a given walkingarea $w \in \mathcal{W}$. Second, the mapping $\text{AllowedButtons}(c)$ defines the set of push buttons that are logically associated with a specific pedestrian crossing $c \in \mathcal{C}$. The set of valid push buttons $\mathcal{B}_{\text{valid}}$ is obtained as the intersection of these two mappings, thereby preventing the activation of buttons that are either physically unreachable or unrelated to the intended crossing movement of pedestrians. If at least one pedestrian satisfies these conditions, the corresponding push button request state is marked as active.

To ensure compatibility with the LISA+[®] signal logic, this persistent state is converted into an input using rising edge detection. A push button request is registered only when

the pedestrian activity changes from inactive to active. Once registered, the request is latched and remains active, reflecting the request storage behaviour.

Latched push button requests are cleared only after the associated pedestrian movement has been served by the LISA+[®] logic, as indicated by a service signal provided by the signal control logic. The algorithm thus generates a vector of binary pedestrian push button inputs $\mathbf{u}(t) = \{u_b(t)\}_{b \in \mathcal{B}}$, where each element corresponds to a predefined push button index. Analogous to single loop detector inputs, pedestrian push button inputs are encoded in field 4 of the OMTC-based input message according to their 0-based indices, ensuring consistent ordering and interpretation within the LISA+[®] logic.

Radio beacon contacts and R09-telegrams

Radio beacon contacts (*Meldepunkte* in German) are commonly used to support public transport priority in LISA+[®]-based signal control systems in the DACH region. When a public transport vehicle equipped with a transmitter passes a radio beacon, a controller-level contact input is generated and transmitted to the signal controller in order to trigger predefined priority actions. A radio beacon contact is activated with an input of 1 when an eligible vehicle passes the beacon location and is mapped to the corresponding index in field 4 of the OMTC-based input message. Otherwise, the input value remains -1.

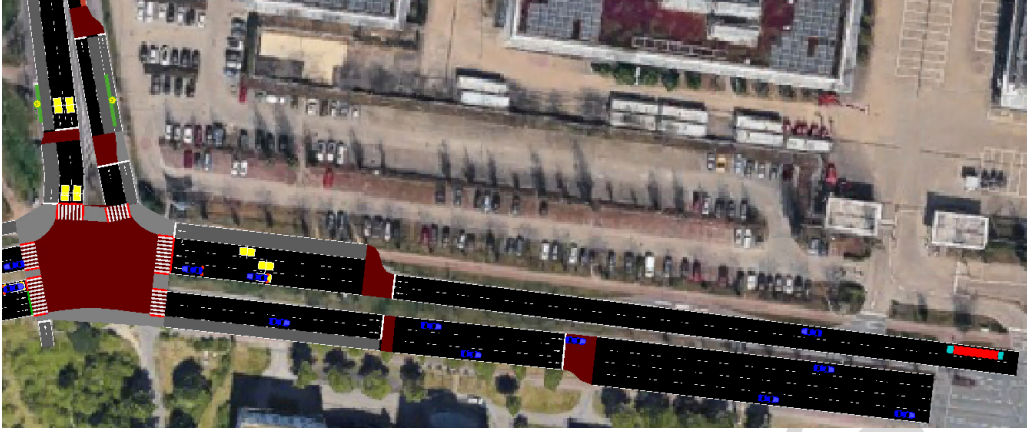
As summarised in Tab. 4, we recommend modelling radio beacon contacts in SUMO using E2 detectors. In the considered configurations, radio beacon activations persist for approximately 3-4 s. This modelling choice ensures a close correspondence between simulated beacon-contact behaviour and field implementations at the controller-input level, as shown in Fig. 7.

R09-telegrams are standardised data messages to transmit public transport information to traffic signal controllers or to the LISA+[®] signal logic in SUMO, as summarised in Tab. 5. All R09-telegram fields are defined manually, with the exception of the field *Schedule deviation*, which is computed dynamically during the simulation. The complete R09-telegram is subsequently encoded and transmitted to the LISA+[®] logic via field 5 of the OMTC-based input message. The *Schedule deviation* is calculated at the moment when a public transport vehicle is detected by the radio beacon and is defined as:

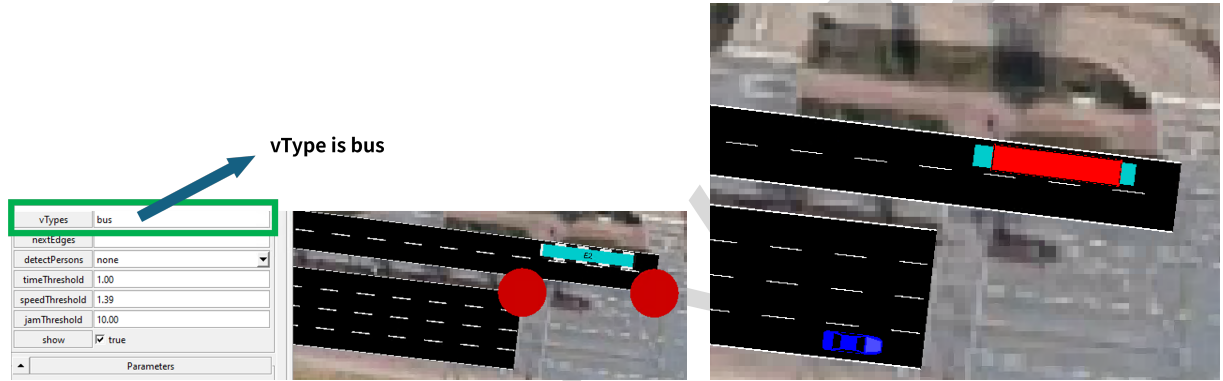
$$\text{Schedule deviation} = t_{\text{beacon}} + \text{Time-to-Intersection} - t_{\text{schedule}}, \quad (1)$$

where t_{schedule} denotes the scheduled timetable time defined by the attribute `until` of the corresponding public transport vehicle in the `*.rou.xml` file. The attribute `until` is interpreted as the scheduled timetable time at the associated public transport stop, reflecting the common practice in DACH public transport systems where a single timetable time is specified per stop. The variable t_{beacon} represents the simulation time at which the public transport vehicle is detected by the radio beacon (corresponding E2 detector). Formally, when the detected vehicle is classified as a public transport vehicle, determined via `traci.vehicle.getVehicleClass(traci.lanearea.getLastStepVehicleIDs(beaconID))`, the beacon detection time is obtained as $t_{\text{beacon}} = \text{traci.simulation.getTime()}$. *Time-to-Intersection* is a default value in the logic, which can be modified by users. The completed R09-telegram is manually constructed via the TraCI interface.

Note that the TraCI functions `traci.vehicle.getStopDelay(vehID)` and `traci.vehicle.getStopArrivalDelay(vehID)` are not used to assign the schedule



(a) Layout of the example signalised intersection in Leipzig, Germany. Radio beacons are typically located upstream of the intersection at a distance of approximately 200m.



(b) Configuration of a *vType*-restricted E2 detector in SUMO for modelling radio beacon contacts. By defining the attribute *vType*, the detector is triggered by public transport vehicles.

(c) A bus (red square) detected by a radio beacon contact, modelled as an E2 detector (light blue square) in SUMO.

Figure 7. Modelling of radio beacon contacts for public transport priority in SUMO.

deviation value in our platform. These functions return the expected departure or arrival delay at the next scheduled stop, respectively, and therefore refer to the temporal deviation at the stop location rather than at the radio beacon position. Since the R09-telegram requires the schedule deviation at the moment of beacon detection, the direct computation in Eq. (1) provides a more accurate representation.

Overall, at each simulation step, the LISA+[®] runtime receives an example OMTC-based input message, such as `1 30 1 0:30{"Run"}{4;0;0;0;1;6;1;2;1;2;0}{(0)1/(1)-1/...}{640187/84/...}{30}{}`, and generates a corresponding OMTC-based output message, such as `30:{30.0}{0;30;1;6;1;0;2;1;2;1}{48/3/3/3/...}{0/0/0/...}{0/0/1/...}{394/3010/0/...}`.

5 Comparison with *ocit2SUMO*

At this example intersection in Leipzig, OCIT[®]-O files are available and can be used to generate traffic signal control logic in SUMO via the *ocit2SUMO* tool [7]. This provides an alternative way of implementing original signal control logic within SUMO. However, the mapping of individual connections and signal groups requires manual completion by appending a specific Bemerkung attribute to the OCIT[®]-O files. Furthermore, the assignment of OCIT[®]-O signal colours to the corresponding signal groups is prone to

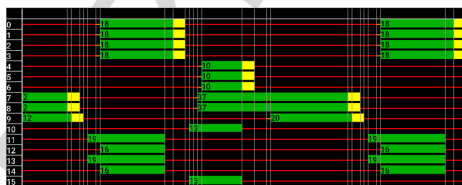
errors during the mapping process. As a result, manual verification and correction are required before use. During processing, transition phases are further divided into sub-phases, which also requires manual adjustment of the phase sequence.

Although the *ocit2SUMO* tool is open-source and simplifies the implementation of original signal control logic in SUMO by using available logic files, the resulting signal timing plans can become highly complex at large or structurally complex intersections. We therefore compare our platform with an *ocit2SUMO*-based platform under identical traffic conditions. The comparison is carried out across five key dimensions: signal logic representation, detector and telegram integration capability, post-processing workload, simulation speed and cost, as summarised in Tab. 10.

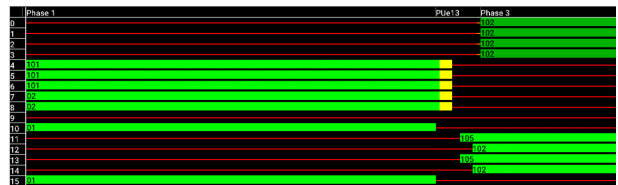
Table 10. Comparison of PyLiSuM- and *ocit2SUMO*-based platforms.

Criteria	PyLiSuM	<i>ocit2SUMO</i>
Signal logic representation	Runtime execution of imported LISA+ [®] logic files	Translation of OCIT [®] -O logic files into SUMO signal definitions
Detector and telegram integration capability	Mapping of controller-level detector states, state transitions, button requests, beacon contacts and R09-telegrams defined in the LISA+ [®] logic files	Limited native support for event-level detector and telegram inputs
Post-processing workload	Lower; mapping configuration is required, but no manual phase reconstruction is needed	Higher; manual completion, verification and correction are required
Simulation speed	Lower; runtime communication with the LISA+ [®] runtime is required	Higher; signal logic is translated into SUMO-internal signal definitions before runtime
Cost	Requires access to the LISA+ [®] commercial software suite	Open-source, but may require higher manual configuration effort

The visual appearance of the two tools during live SUMO simulations is shown in Fig. 8. The comparison illustrates the different execution principles of the two platforms. *PyLiSuM* executes the original LISA+[®] logic at runtime and drives it with controller-level detector, button, beacon and telegram inputs generated from the SUMO simulation. This allows the signal states to change according to the imported traffic-actuated logic and the simulated input events. In contrast, *ocit2SUMO* translates the available OCIT[®]-O logic into SUMO signal control definitions before runtime. This results in a more compact SUMO-internal representation, but provides less direct support for event-level detector and telegram inputs. The visual comparison therefore serves as a qualitative illustration of the runtime behaviour and integration differences between the two platforms.



(a) PyLiSuM during a live SUMO simulation



(b) *ocit2SUMO* during a live SUMO simulation

Figure 8. Runtime visualisation of PyLiSuM and *ocit2SUMO* in live SUMO simulations under the same traffic scenario.

6 Applications and Discussion

We discuss typical application scenarios of our platform and highlight how it can be used in both engineering practice and research.

As a virtual acceptance platform for urban traffic signal control projects

In many cities in the DACH region, traffic signal control systems must be validated against defined operational requirements before they are accepted for use in the field. By enabling the execution of original LISA+[®] logic within SUMO, our platform provides a reproducible and transparent simulation environment that can support virtual acceptance and pre-deployment testing. Traffic engineers and urban traffic authorities can test detector configurations, public transport priority behaviour and signal timing logic under simulated traffic demand without relying on proprietary controller hardware or costly field tests. This makes our platform a suitable virtual platform to support acceptance procedures for urban traffic signal control projects.

As a baseline for developing advanced signal control algorithms

A realistic and practice-oriented signal control baseline is important for the development and evaluation of advanced signal control algorithms. The middleware *PyLiSuM* allows direct comparison between new control approaches and existing LISA+[®]-based signal control logic under the same traffic conditions. Researchers can use LISA+[®] logic as a baseline that represents current practice in many DACH cities. This enables fair comparisons and supports the step-by-step development of advanced signal control algorithms that build on proven control logic rather than replacing it entirely.

As a safety layer for reinforcement learning-based control

Reinforcement learning (RL) methods for traffic signal control often raise concerns about safety, transparency, and compliance with operational rules. The middleware *PyLiSuM* can be used as a safety layer that contains original LISA+[®]-based signal control logic, helping to preserve basic safety and operational constraints. In such a combined approach, an RL agent may generate control actions or parameter changes, while the underlying LISA+[®] logic enforces hard constraints and fallback behaviour. This setup allows researchers to explore learning-based optimisation while keeping traffic signal operation safe.

Studying the placement of radio beacons for public transport priority

The placement of radio beacons has a strong influence on the performance of public transport priority in LISA+[®]-based signal control systems. Beacon locations determine when priority requests are triggered and how much time the controller has to respond before a public transport vehicle reaches the intersection. By modelling radio beacon contacts and computing the *Schedule deviation* at the moment of detection, our platform allows a systematic study of different beacon placement options. Researchers and practitioners can examine how upstream distance, traffic conditions, and timetable deviations affect priority performance and impacts on general traffic.

7 Conclusion

We present a SUMO- and Python-based platform that enables the execution of complete LISA+[®] traffic signal control logic within SUMO via *PyLiSuM*. The platform models LISA+[®]-compatible controller-level inputs from loop-detector states and state transitions, pedestrian push buttons, radio beacon contacts and R09-telegrams, thereby reproducing LISA+[®]-based traffic signal control in SUMO without rewriting the original controller logic.

Another highlight of this work is the modelling of R09-telegrams. Particular attention is given to public transport priority, where the *Schedule deviation* is computed dynamically at the moment of radio beacon detection, reflecting common practice in LISA+[®]-based public transport priority configurations. This enables a realistic evaluation of public transport priority behaviour in simulation.

Overall, this platform is practical and transparent for both engineering practice and research. It can support virtual testing and acceptance preparation for urban traffic signal control projects, serve as a practice-oriented baseline for the development of advanced signal control algorithms, and enable further studies on public transport priority and detector or beacon placement. Future work will focus on extending support to additional controller types, and extending the approach from single intersection control to the network level.

Data availability statement

All traffic signal provisioning files used in this work are owned by and provided by the Mobilitäts- und Tiefbauamt (MTA; Mobility and Civil Engineering Office) of the City of Leipzig. Due to usage agreements, these files cannot be made publicly available as part of this work. *PyLiSuM* is publicly available at <https://github.com/TU-Dresden-VPA/PyLiSuM>.

Author contributions

Markus Runhao Zhou: Conceptualisation, Methodology, Software, Validation, Formal analysis, Investigation, Resources, Data Curation, Writing – Original Draft, Visualisation, Project administration. **Johann Krüger:** Software, Writing – Original Draft. **Meng Wang:** Writing – Review & Editing, Supervision, Project administration, Funding acquisition.

Competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the Work Package 5 - Upgrading of the Infrastructure within the project ABSOLUT II - Autonomous On-Demand Shuttles Providing Individual Public

Transport Services for the Suburban Areas of Leipzig, funded by the Bundesministerium für Forschung, Technologie und Raumfahrt der Bundesrepublik Deutschland (BMFTR; Federal Ministry of Research, Technology and Space and of Germany) under Grant No. 01ME23001B.

Acknowledgements

The authors sincerely thank colleagues from the Mobilitäts- und Tiefbauamt (MTA; Mobility and Civil Engineering Office) of the City of Leipzig for their outstanding support.

The authors are solely responsible for the content.

References

- [1] Schlothauer & Wauer Ingenieurgesellschaft für Straßenverkehr mbH, *LISA+*, [Accessed: 2025-11-14]. [Online]. Available: <https://www.schlothauer.de/en/software-lisa>.
- [2] P. Alvarez Lopez, M. Behrisch, L. Bieker-Walz, et al., "Microscopic traffic simulation using SUMO," in *The 21st IEEE International Conference on Intelligent Transportation Systems (ITSC)*, Maui, HI, USA: IEEE, 2018, pp. 2575–2582. doi: [10.1109/ITSC.2018.8569938](https://doi.org/10.1109/ITSC.2018.8569938).
- [3] M. Barthauer and B. Friedrich, "External signal control: Integrating LISA+ into SUMO," in *SUMO 2017 – Towards Simulation for Autonomous Mobility*, vol. 31, Deutsches Zentrum für Luft-und Raumfahrt e.V., 2017, pp. 143–154.
- [4] M. Barthauer and B. Friedrich, "Connecting microscopic traffic simulation and LISA+ external signal control," *Transportation Research Procedia*, vol. 27, pp. 420–427, 2017, issn: 23521465. doi: [10.1016/j.trpro.2017.12.007](https://doi.org/10.1016/j.trpro.2017.12.007).
- [5] M. Bottazzi, L. C. Touko Tcheumadjeu, J. Trumpold, J. Erdmann, and R. Oertel, "LiSuM: Design and development of a middleware to couple virtual LISA+ TLS controller and SUMO simulation," in *SUMO 2017 – Towards Simulation for Autonomous Mobility*, vol. 31, Deutsches Zentrum für Luft-und Raumfahrt e.V., 2017, pp. 179–192. [Online]. Available: <https://elib.dlr.de/112830/>.
- [6] Verband Deutscher Verkehrsunternehmen (VDV), *Technical Requirements for Automatic Vehicle Location/Control Systems - Radio Data Transmission (BON Version)*, Cologne, Germany, 1992.
- [7] J. Erdmann, *SUMO-OCIT: Tools for using OCIT data with Eclipse SUMO*, [Accessed: 2025-12-26], 2025. [Online]. Available: <https://github.com/DLR-TS/sumo-ocit>.
- [8] A. Müller, T. Ferfers, F. Hufen, et al., "KI4LSA – Artificial intelligence for traffic signal control. Joint final report [KI4LSA – Künstliche intelligenz für Lichtsignalanlagen. Gemeinsamer Abschlussbericht]," Fraunhofer IOSB, Technical Report, 2024, [In German. Accessed: 2026-01-10]. [Online]. Available: <https://publica.fraunhofer.de/entities/publication/56f97ec6-a9d5-4b13-8856-3ab96d259d2f>.
- [9] *Open Communication Interface for Road Traffic Control Systems – Outstations: Traffic signal controllers*, OCIT-O_TSC_V3.0_A02, [Accessed: 2026-01-14], OCIT Developer Group (ODG) & Partner, 2024. [Online]. Available: https://www.ocit.org/media/ocit-o_tsc_v3.0_a02_en.pdf.
- [10] R. Fielding and J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content," Internet Engineering Task Force (IETF), Request for Comments RFC 7231, 2014, [Accessed: 2026-01-14]. [Online]. Available: <https://www.rfc-editor.org/info/rfc7231>.